

CHƯƠNG 3 – DECORATOR PATTERN

Head First Design Patterns

Slide 1 — Trang bìa

DECORATOR PATTERN

Head First Design Patterns – Chapter 3

- Môn học: Phân tích & Thiết kế phần mềm
- Sinh viên: [Tên sinh viên]
- Giảng viên hướng dẫn: [Tên GV]
- Ngày thuyết trình: [Ngày]

Gợi ý hình:

- Coffee shop
 - Layer/wrapper
 - Decorate icon
-

Slide 2 — Nội dung trình bày

Nội dung

1. Vấn đề của Inheritance
 2. Open-Closed Principle
 3. Decorator Pattern
 4. UML Diagram
 5. Ví dụ Starbuzz Coffee
 6. Cài đặt bằng Java
 7. Ưu và nhược điểm
 8. Ứng dụng thực tế
-

Slide 3 — Vấn đề của Inheritance

Tại sao cần Decorator Pattern?

Khi dùng kế thừa để mở rộng chức năng:

- Số lượng class tăng rất nhiều
- Khó bảo trì hệ thống

- Khó mở rộng
- Dễ trùng lặp code

Ví dụ:

- EspressoWithMilk
- EspressoWithMilkMocha
- EspressoWithMilkMochaWhip

Kết quả:

→ “Class Explosion”

Slide 4 — Open Closed Principle

Open-Closed Principle

“Open for Extension, Closed for Modification”

Ý nghĩa:

- Có thể mở rộng chức năng
- Không cần sửa code cũ
- Hạn chế lỗi phát sinh

Ví dụ:

- Thêm topping mới cho coffee
- Không sửa class Beverage

Decorator Pattern hỗ trợ rất tốt nguyên lý này.

Slide 5 — Decorator Pattern là gì?

Decorator Pattern

Định nghĩa

Decorator Pattern cho phép:

- Thêm chức năng cho object tại runtime
- Không sửa class gốc
- Linh hoạt hơn inheritance

Ý tưởng

Coffee

- Milk
 - Mocha
 - Whip
-

Slide 6 — Ý tưởng hoạt động

Cách hoạt động

Concrete Component ↓ Decorator ↓ Decorator ↓ Decorator

Ví dụ:

Espresso → MilkDecorator → MochaDecorator → WhipDecorator

Đặc điểm

- Wrapper object
 - Runtime extension
 - Composition over inheritance
-

Slide 7 — UML Diagram

UML Structure

Thành phần chính

Component

- Interface/base class chung

ConcreteComponent

- Object gốc

Decorator

- Wrapper class

ConcreteDecorator

- Thêm behavior mới

Flow

Client → Decorator → ConcreteComponent

Slide 8 — Ví dụ Starbuzz Coffee

Starbuzz Coffee

Component

- Beverage

Concrete Component

- Espresso
- HouseBlend

Decorator

- CondimentDecorator

Concrete Decorator

- Mocha
 - Soy
 - Whip
-

Slide 9 — Code Beverage

Beverage Class

```
public abstract class Beverage {  
    String description = "Unknown Beverage";  
  
    public String getDescription() {  
        return description;  
    }  
  
    public abstract double cost();  
}
```

Ý nghĩa

- Base abstract class
 - Tất cả đồ uống kế thừa từ đây
-

Slide 10 — Code Decorator

Decorator Base Class

```
public abstract class CondimentDecorator extends Beverage {  
    public abstract String getDescription();  
}
```

Ý nghĩa

- Decorator cũng là Beverage
- Có thể wrap Beverage khác

Highlight

“Decorator IS-A Beverage”

Slide 11 — Concrete Decorator

Ví dụ Mocha Decorator

```
public class Mocha extends CondimentDecorator {  
  
    Beverage beverage;  
  
    public Mocha(Beverage beverage) {  
        this.beverage = beverage;  
    }  
  
    public String getDescription() {  
        return beverage.getDescription() + ", Mocha";  
    }  
  
    public double cost() {  
        return beverage.cost() + 0.20;  
    }  
}
```

Ý nghĩa

- Wrap object cũ
 - Thêm behavior mới
 - Gọi lại method object gốc
-

Slide 12 — Client sử dụng

Client Code

```
Beverage beverage = new Espresso();

beverage = new Mocha(beverage);
beverage = new Whip(beverage);

System.out.println(
    beverage.getDescription()
    + " $" + beverage.cost()
);
```

Flow

Espresso → Mocha → Whip

Slide 13 — Kết quả hoạt động

Output

```
Espresso, Mocha, Whip $2.49
```

Giải thích

- Behavior được cộng dồn runtime
 - Mỗi decorator thêm chức năng mới
-

Slide 14 — Ưu điểm

Ưu điểm

- Linh hoạt
 - Runtime extension
 - Giảm subclass
 - Dễ mở rộng
 - Tuân thủ Open-Closed Principle
 - Reusable
-

Slide 15 — Nhược điểm

Nhược điểm

- Nhiều object nhỏ
 - Debug khó hơn
 - Hệ thống phức tạp nếu lạm dụng
 - Khó theo dõi luồng object
-

Slide 16 — Ứng dụng thực tế

Ứng dụng thực tế

Java I/O

- FileInputStream
- BufferedInputStream
- DataInputStream

Framework

- ExpressJS Middleware
 - Spring Security
 - Logging System
 - UI Components
-

Slide 17 — Decorator vs Inheritance

So sánh

Inheritance	Decorator
Compile-time	Runtime
Rigid	Flexible
Nhiều subclass	Ít subclass
Khó mở rộng	Dễ mở rộng

Kết luận

Decorator linh hoạt hơn inheritance.

Slide 18 — Tổng kết

Tổng kết

Decorator Pattern giúp:

- Mở rộng object linh hoạt
- Giảm subclass explosion
- Tăng khả năng maintain
- Áp dụng composition thay vì inheritance

Design Principle

“Favor Composition Over Inheritance”

THANK YOU

Questions?